



Université de Sherbrooke

SQL

Fonctions et procédures

SQL_06

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

Zeineb ZAIET (zeineb.zaiet@usherbrooke.ca)

(les auteurs sont cités en ordre alphabétique nominal)

—

Scriptorum/Scriptorum/SQL_06-Fonctions-et-Procedures, version 1.0.0.a, en date du 2026-02-14

— document de travail, ne pas citer —

Sommaire

...

Mise en garde

Le présent document est en cours d'élaboration ; en conséquence, il est incomplet et peut contenir des erreurs.

Historique

diffusion	resp.	description
2026-01-03	CK	Récupération de notes diverses.

Table des matières

Introduction.....	4
1. Présentation.....	4
2. Les fonctions	4
2.1. Syntaxe (PostgreSQL)	4
2.2. Exemple (1).....	5
2.3. Exemple (2).....	5
2.4. Exemple (3).....	5
2.5. Exemple (4).....	6
3. Les procédures.....	7
3.1. Syntaxe (PostgreSQL)	7
3.2. Exemple (1).....	7
3.3. Exemple (2).....	8
3.4. Exemple (3).....	9
Conclusion.....	11
Références	12
Définitions	13

Introduction

Le présent document a pour but de présenter une introduction au langage de définition de fonctions et de procédures qu'on retrouve dans SQL.

Dans un premier temps, nous présenterons une grammaire simplifiée, sous-ensemble propre de la grammaire de la norme ISO 9075:2016, avec, au besoin, certaines particularités de PostgreSQL.

1. Présentation

- Une fonction calcule une valeur suite à un appel sur la base de ses paramètres ou de l'état du calculateur, sans en changer l'état.
- Une procédure change l'état de la BD suite à un appel sur la base de ses paramètres et de l'état du calculateur.

2. Les fonctions

2.1. Syntaxe (PostgreSQL)

```
function_def ::=
  CREATE [ OR REPLACE ] FUNCTION name ( [ argument [, ...] ] )
    RETURNS ret_type
    function_body

argument ::=
  [ arg_mode ] [ arg_name ] arg_type [ DEFAULT default_expr ]

arg_mode ::=
  IN | VARIADIC

ret_type ::=
  type_denotation | TABLE ( [ column_name column_type [, ...] ] )
```

arg_mode : en cas d'omission, le mode est IN. Une fonction ne devrait utiliser que des arguments IN ou VARIADIC. VARIADIC, c'est un tableau d'argument du même type.

ret_type : une troisième option, non conforme au standard ISO, est possible SETOF composite_type le composite_type est celui des tuples d'une table préalable créée (CREATE TABLE) ou celui défini par un CREATE TYPE.

```
function_body ::=
  RETURN expression
| BEGIN ATOMIC [statement ;] ... END
| LANGUAGE lang_name AS '$$' string '$$'
```

La routine, aussi appelée permanently stored module (PSM), est un objet stocké dans la base de données, comme les domaines, les types, les tables et les vues.

Le corps (body) d'une routine est la description textuelle du traitement dans un langage (lang_name) spécifié.

Les langages utilisables varient d'une dialecte à un autre ; comprennent minimalement SQL ; comprennent souvent Pascal, C, C++, Java et Python ; comprennent souvent un « PSM language ».

Chaque dialecte SQL semble avoir défini son propre PSM language plutôt que d'adhérer à celui prescrit par la norme ISO : plpgsql est celui de PostgreSQL ; PL/SQL est celui d'Oracle.

2.2. Exemple (1)

Sous-type

Le numéro de téléphone ne comprend que les chiffres qui le composent (aucune marque d'édition telle que tirets, parenthèses, points, etc.). Les numéros des pays membres de l'ITU comprennent entre 8 et 13 chiffres. Les chiffres des numéros canadiens et états-uniens se répartissent ainsi : indicatif national : 1; indicatif régional : 3; équipement : 7.

```
create domain Telephone as
  varchar(13)
  constraint Telephone_ITU check(value similar to '[0-9]{8,13}');

create domain Telephone_CA as
  Telephone
  check (value similar to '[1][0-9]{10}');
```

Définition Indicatif Regional

```
create function indicatifRegional(t Telephone_CA)
  returns VARCHAR(3)
  return substr(t, 2, 3);
```

Utilisation Indicatif Regional

```
select IndicatifRegional('18191234567');
```

2.3. Exemple (2)

Identifier les activités du département d'informatique.

Définition

```
create or replace function
  estActiviteInfo(sigle SigleCours)
  returns boolean
  language SQL as
  $$
  select substr(sigle, 1, 3) in ('IFT', 'IGE', 'IGL', 'IMN')
  $$;
```

Utilisation

```
select sigle, estActiviteInfo(sigle)
from activite;
```

2.4. Exemple (3)

Calculer la côte d'une note.

Définition SQL

```
create or replace function
```

```

    calculerCote(_note numeric(3))
    returns char(1)
    language SQL as
$$
select case
    when _note >= 90 then 'A'
    when _note between 80 and 89 then 'B'
    when _note between 60 and 79 then 'C'
    when _note between 40 and 59 then 'D'
    else 'E'
end
$$;

```

Définition plpgsql

```

create or replace function
    calculerCote(_note numeric(3))
    returns char(1)
    language plpgsql as
$$
declare
    cote char(1);
begin
    if _note >= 90 then cote := 'A';
    elsif _note between 80 and 89 then cote := 'B';
    elsif _note between 60 and 79 then cote := 'C';
    elsif _note between 40 and 59 then cote := 'D';
    else cote := 'E';
    end if;
    return cote;
end
$$;

```

Utilisation

```

with note_finale as (
    select matricule,
           activite,
           sum(note) as noteF
    from Resultat
    group by matricule, activite
)
select matricule, activite, calculerCote(noteF) as cote
from note_finale;

```

2.5. Exemple (4)

Calculer le bulletin d'une personne étudiante.

Définition SQL

```

create or replace function
    calculerBulletin(_matricule universite.matricule)
    returns Table(activite universite.siglecours,
                 trimestre universite.trimestre,
                 note universite.note,
                 cote char(1))
    language SQL as
$$

```

```
-- Hypothèse toutes les notes sont sur 100
with note_activite as
(
    select activite, trimestre, avg(note) as note
    from resultat
    where matricule = _matricule
    group by activite, trimestre
)
select activite, trimestre, note, calculerCote(note) cote
from note_activite;
$$;
```

Utilisation

```
select *
from calculerBulletin('15110132');
```

3. Les procédures

3.1. Syntaxe (PostgreSQL)

```
procedure_def ::=
CREATE [ OR REPLACE ] PROCEDURE name ( [ argument [, ...] ] )
{ other_procedure_qual } ...
procedure_body

procedure_body ::=
BEGIN ATOMIC [statement ;] ... END
| LANGUAGE lang_name AS string
```

...

3.2. Exemple (1)

Création de types d'évaluation :

1. Créer une fonction pour générer un acronyme du type d'évaluation.
L'acronyme est formé de la première lettre de chaque mot dans une chaîne de caractères.
Si la chaîne de caractère est formée d'un seul mot l'acronyme est formé des deux premières lettres.
2. Créer une procédure d'insertion d'un type d'évaluation

Définition fonction

```
create or replace function
    acronyme(_description varchar)
    returns char(2)
    language SQL as
$$
select case
    when count(mot) > 1 then upper(string_agg(left(mot, 1), ''))
    when count(mot) = 1 then upper(string_agg(left(mot, 2), ''))
end
from regexp_split_to_table(_description, '\s+') AS mot;
$$;
```

Définition procédure

```
create or replace procedure
  typeEvaluation_ins(_description varchar)
  language SQL as
$$
insert into TypeEvaluation(code, description)
  values (acronyme(_description), _description);
$$;
```

3.3. Exemple (2)

Inscription des personnes étudiantes en informatique.

1. Créer une table pour consigner l'information des inscriptions.
2. Créer une procédure pour inscrire une personne étudiante à tous les activités d'informatique.

Modification du schéma

```
create table Inscription
(
  matricule Matricule not null,
  sigle      SigleCours not null,
  trimestre Trimestre not null,
  constraint Inscription_cc0 primary key (matricule, sigle, trimestre),
  constraint Inscription_cr0 foreign key (matricule) references Etudiant (matricule),
  constraint Inscription_cr1 foreign key (sigle) references Activite (sigle)
);
```

Définition procédure SQL

```
create or replace procedure
  inscrireInfo(_matricule Matricule,
              _nom varchar(30),
              _adresse varchar(30),
              _trimestre Trimestre)
  language SQL as
$$
insert into Etudiant(matricule, nom, adresse)
  values (_matricule, _nom, _adresse);

insert into Inscription(matricule, sigle, trimestre)
  select _matricule, sigle, _trimestre
  from Activite
  where estActiviteInfo(sigle);
$$;
```

Définition procédure plpgsql

```
create or replace procedure
  inscrireInfo(_matricule Matricule,
              _nom varchar(30),
              _adresse varchar(30),
              _trimestre Trimestre)
  language plpgsql as
$$
declare
  found int;
```

```

begin
  select count(*) into found
  from Etudiant
  where matricule = _matricule;

  if found = 0 then
    insert into Etudiant(matricule, nom, adresse)
    values (_matricule, _nom, _adresse);
    raise notice 'Ajout Etudiant %', _matricule;
  end if;

  insert into Inscription(matricule, sigle, trimestre)
  select _matricule, sigle, _trimestre
  from Activite
  where estActiviteInfo(sigle)
  limit 5;
end
$$;

```

Utilisation procédure

```
call InscrireInfo('20122334', 'Jeanne', 'Sherbrooke', '20261');
```

3.4. Exemple (3)

Archiver les activités qui ne sont plus enseignées depuis plus que deux ans.

1. Documenter le statut de l'activité.
Ajouter un attribut pour représenter le statut {1=active | 0=archive}.
2. Créer une fonction pour archiver une activité qui consiste à affecter le statut 0 à un sigle.
3. Créer une procédure pour archiver les activités 2 ans et plus.

Modification schéma

```

alter table Activite
  add column statut char(1) not null default 1;

```

Définition archiver une activité

```

create or replace procedure
  archiver_activite(_sigle siglecours)
  language SQL as
$$
  update activite
  set statut = 0
  where sigle = _sigle;
$$;

```

Définition archiver des activités

```

create or replace procedure
  archiver_activite_2ans()
  language plpgsql as
$$
declare

```

```
rec record;  
begin  
  for rec in select sigle  
    from activite as r1  
    where (extract('year' from current_date)::int - 2)  
      >= all (select substr(trimstre, 1, 4)::int  
        from Resultat as r2  
        where r1.sigle = r2.activite)  
  loop  
    call archiver_activite(rec.sigle);  
  end loop;  
end  
$$;
```

Utilisation

```
call archiver_activite_2ans();
```

Conclusion

Références

[Elmasri2016]

Ramez ELMASRI et Shamkant B. NAVATHE;
Fundamentals of database systems;
7th Edition, Pearson, Hoboken (NJ, US), 2016;
ISBN 978-0-13-397077-7.

[Date2015a]

Chris J. DATE;
SQL and relational theory: how to write accurate SQL code;
O'Reilly Media, 2015;
ISBN 978-1-4919-4117-1.

PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

Définitions

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-10-06 21:20:36 UTC



Université de Sherbrooke